Security Issues in Outsourced XML Databases

Tran Khanh Dang
National University of Ho Chi Minh City, Vietnam

ABSTRACT

In an outsourced XML database service model, organizations rely upon the premises of external service providers for the storage and retrieval management of their XML data. Since, typically, service providers are not fully trusted, this model introduces numerous interesting research challenges. Among them, the most crucial security research questions relate to data confidentiality, user and data privacy, query assurance, secure auditing, and secure and efficient storage model. Although there exists a large number of related research works on these topics, we are still at the initial stage and the research results are still far from practical maturity. In this chapter, we extensively discuss all potential security issues mentioned above and the existing solutions, and present open research issues relevant to security requirements in outsourced XML databases.

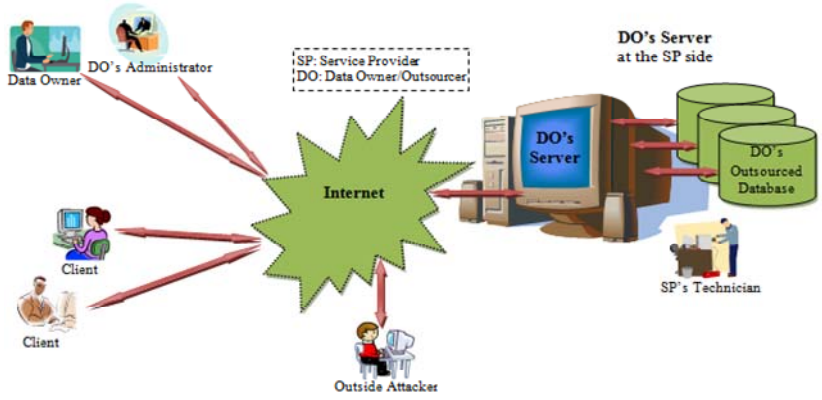
1. INTRODUCTION

In the early days of ubiquitous computing and up through the late 1990s, owning and operating a sophisticated database was a strategic advantage for an organization (Bostick, 2008). Nowadays, however, nearly every organization possesses some sort of database containing different valuable information, which is considered the lifeblood of the organization. The ability to store data is therefore no longer a strategic advantage in and of itself. Recent investigations also report that storage capacity and security requirements will soon become a big problem for organizations if they still want to manage large in-house databases (Gantz, 2007). The management of the database infrastructure and the data needs has become much more of a commodity (IBM, 2008a; DBADirect, 2008). As a result, the value is not in owning a database but in how it is used and what can be got from it.

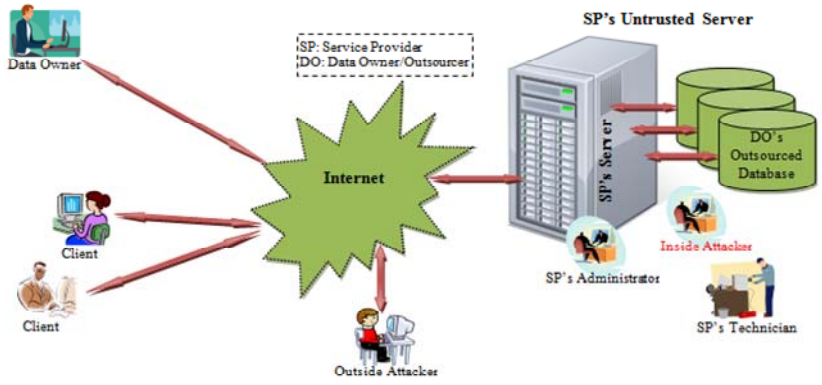
Recently, with rapid developments of the Internet and advances in the networking technologies, outsourcing database services has been emerging as a promising trend to overcome the limitations of the in-house database storage model (Hacigümüs et.al., 2002b; Bouganim & Pucheral, 2002). Basically, there are two kinds of database outsourcing models: *housing-based* and *hosting-based*. With the *housing-based* database outsourcing model, the server and data are the property of the *outsourcer* (i.e., the data owner as illustrated in Figure 1a) and the outsourcer installs the servers. In this case, the outsourcing service provider provides the *physical* security of machines and data, and monitors (and if necessary restores) the operating condition of the server. Of course, the service provider protects the outsourced data against *physical attacks* both from **outside** and **inside**. Notwithstanding, the access rights of the service provider depend on a particular contract with its client (i.e., the outsourcer). Basically, the service provider only

gets *a special account* with the server for special managerial purposes, but the outsourcer can determine the necessary access rights of that account. As for the *hosting-based* database outsourcing model, instead of keeping data in local servers, accommodated internally inside an organization, and having a professional technical team to manage the relevant database services such as software and hardware updates, server performance tuning, or security and authorization management, etc., now all data management needs can be outsourced to outside database service providers (see Figure 1b). In this case, the service provider provides all needed facilities, such as hardware, operating system, etc. to host and manage the outsourced data. In particular, it is different from the housing-based model in that the database administrator in this hosting-based model belongs to the service provider, but not the outsourcer's. By employing this hosting-service outsourcing model, usually called the outsourced database service (ODBS) model (Dang, 2005; Hacigümüs et. al., 2002b), organizations have more freedom to concentrate on and invest in their core business activities. The ODBS model is obviously preferable to the housing-based one. In both models, however, a service provider is typically not fully trusted, and thus *they raise numerous interesting research challenges related to security issues* (Hacigümüs et.al., 2002a; Smith & Safford, 2001; Dang, 2008; Damiani et al., 2003; Dang, 2006a; Dang, 2006b; Narasimha & Tsudik, 2006; Sion, 2005; Du & Atallah, 2000; Pang & Tan, 2004; Thuraisingham, 2005; etc.). To make the outsourcing model full-fledged and practically applicable, security-related issues must be addressed radically.

Figure 1. Two database-outsourcing models: housing-based (a); and hosting-based (b).



(a) The housing-based database outsourcing model



(b) The hosting-based database outsourcing model

Moreover, data management needs constitute the essential enabling technology for scientific, engineering, business, and social communities (Carminati & Ferrari, 2006). In order to facilitate the data exchange efficiently across heterogeneous systems, organizations should use a common standard data model/language like EDI (Electronic Data Interchange), XML (eXtensible Markup Language), etc. Among such data models/languages, XML has been widely accepted and adopted as the basis for developing a variety of modern application domains/architectures (W3C, 2008; Brinkman et al., 2004; Bertino et.al., 2007). Emerging simultaneously with the important new database outsourcing trend above, i.e., the ODBS model, *security issues in outsourced XML databases* have been and will be among the most active topics in the research community as well as commercial world. Those security issues range from fundamental to advanced topics in the information security area, namely *data confidentiality*, *user and data privacy*, *query assurances and quality of services*, *secure auditing*, and *secure and efficient storage*. In this chapter, for convenience, we call the outsourced XML database service model, i.e., the XML data are outsourced to the service provider using the hosting-based model, the *OXMLDBS* model. This chapter will discuss all known security issues and existing solutions, and will cover the open issues relevant to security requirements in outsourced XML databases. Although we will focus our discussions on the OXMLDBS (and ODBS) model, a brief discussion about potential security issues in the housing-based database outsourcing model will also be given in section 2.

The rest of this chapter is organized as follows: Section 2 presents background knowledge, security issues, and related work in the area of ensuring data outsourcing security. Most of existing approaches to security issues in outsourced XML databases are based on the proposed solutions and mechanisms in this area. Section 3 presents and discusses state-of-the-art approaches dealing with security issues in outsourced XML databases. Section 4 introduces open research issues and future trends. Finally, section 5 provides discussions of the overall coverage of the chapter and concluding remarks.

2. BACKGROUND

2.1. Security Issues in Outsourced Database Services

According to the two outsourcing models as illustrated in Figure 1, each of them in general consists of three entities: (1) the data owner(s)/outsourcer(s), (2) the database service provider, and (3) the client(s). The data owner creates, modifies and deletes the contents of the database. The server hosts the owner's database. At the server side, for both models, there are technicians responsible for *normal* technical maintenance tasks *except for* the database administration, which is under the control of some database administrator(s). The database administrator belongs to the outsourcer in the housing-based outsourcing model, while they are the service provider's manpower in the hosting-based one. This is the major factor that causes many security concerns in the ODBS and OXMLDBS models as we will discuss later. The clients issue queries about the database to the server (Dang, 2006b; Hacigümüs et.al., 2002b). Moreover, in many real-world cases, the data owners are also the unique clients (Du & Atallah, 2000; Dang, 2006b; Narasimha & Tsudik, 2006) (see Figure 3). Interestingly, due to its much more inherent simplicity, most of existing research works limit their focus to this special case (Hacigümüs et.al., 2002b; Damiani et al., 2003; Schrefl et. al., 2005; Li et. al., 2006; Nguyen et. al., 2007; Xie et. al., 2007; etc.). Even then, security issues of concern in the OXMLDBS model are not identical to those of the housing-based model. Below, we discuss the challenging security issues in both interested outsourcing models.

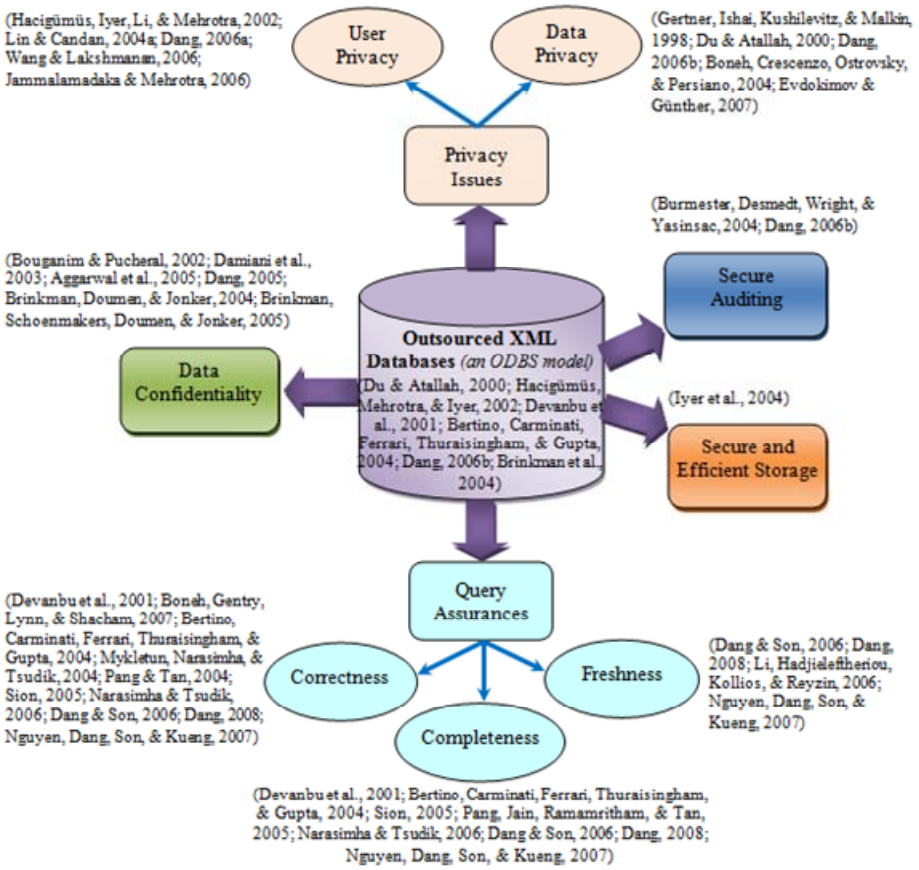
Intuitively, with the housing-based outsourcing model, the data owner is no more responsible for the server accommodation, internet connection, and other infrastructures. Data management-related tasks, however, are still their regular duties. Even then, concerning security issues in the housing service business model, "**soft**" security-related aspects are usually ignored or understood that "*the installed software or the data owner must be responsible for it*". For example, if a data owner chooses this database housing service at a company *M* and *M* gets a special account *A* for some special managerial activities (*A* was assigned very limited rights on the data owner's "outside-housed" database *DB*), then *M* is typically liable only for the so-called **physical** security of the data owner's hardware and data. It means that *M* is responsible only for physically securing the server containing the outsourced data (e.g., power, seamless internet connection, etc), but *not the data contents* from malicious activities (e.g., hacking, inference attacks etc). This is reasonable only if (the client believes that) the DBMS is working as expected in terms of security. In practice, *serious* problems can occur as the DBMS has some security flaws and the client may not be aware of these "soft" flaws before they are discovered by the housing company *M*. In this case, *M* may make use of *A* and the found security flaws to get control (or so) of the client's *DB*, which is currently housed on *M*'s premises. This is an interesting research problem and it will be very much better if *M* can provide the client with a means (tools/software) of detecting the database security flaws and visually monitoring the account *A*'s real-time activities, especially with the possible *DB* security flaws. If *M* can do this, the client will be more than happy because they are assisted in detecting their *DB* security flaws and can control or anticipate the potential harms that may be caused by *A*. Although a lot of research work has been carried out to propose effective mechanisms for detecting database security breaches (Dang et al., 2008), the proposed solutions still do not satisfy users. Much more work will have to be done to secure the outsourcer's DBMSs and data in the housing-based outsourcing model. Nevertheless, in this chapter, we will not go into further detail about the security issues in this model. Next, we discuss security concerns in the ODBS (and OXMLDBS) model, a client-centric classification of the model and crucial security issues in each subclass. It is obvious that in the ODBS (and OXMLDBS) model, the data owner is no longer responsible for data management; rather, it outsources its data to one or more service providers that provide management services and query processing functionalities. Clearly, this method of data outsourcing leads to challenging security issues in that, by outsourcing its data, the data owner may potentially lose control over them. More concretely, both data and users' queries can now be exposed to the server and hackers/malicious users (corresponding to inside and outside attackers as shown in 1b, respectively). Therefore, in this ODBS model, apart from secure network communication channels and other necessary security procedures at the user side (Axelrod, 2004), efficient and effective solutions to security threats inside the server are indispensable. Consequently, a lot of research work is currently being carried out to ensure secure management of data even in the presence of an *untrusted server* (Carminati & Ferrari, 2006; Li et. al., 2006; Dang, 2008; Xie et.al., 2007; Nguyen & Dang, 2008; etc.). We discuss these server-side security-related issues below.

Security issues in the ODBS model and outsourced XML databases. First, because the lifeblood of every organization is the information stored in its databases, making outsourced data *confidential* is therefore one of the foremost challenges in this model. In addition, *privacy-related concerns* must also be taken into account due to their important role in the real-world applications (Thuraisingham, 2005). No less importantly, to make the outsourced database service viable and applicable, the system has to provide users with some means of verifying the *query assurance* claims of the service provider. Last but not least, solutions to secure auditing and efficient storage must also be addressed radically to fulfill major security requirements of concern. Overall, *most crucial* security-related research questions in the ODBS model in common, and for *outsourced XML databases* in particular, relate to the issues below (see Figure 2):

- *Data confidentiality*: Outsiders and the server's operators (database administrator-DBA) cannot see the user's outsourced data contents in any cases (even as the user's queries are performed on the server). Herein, *the user* may be the *data owner/outsourcer* or *its clients*.
- *Privacy concerns*: Although privacy-related issues have been widely investigated (PRIME Project, 2004), the question "what is the complexity of the privacy problem?" is still open in as much as the answer is quite different, depending not only on technology, but also on sociology and politics (Thuraisingham, 2005). In our context, however, user and data privacy issues are clearly identified and defined as follows (Chor et. al., 1995; Gertner et. al., 1998; Burmester et. al., 2004; Dang, 2008):
  - *User privacy*: Users do not want the server and even the DBA to know about their queries and the results. Sometimes, user identities should also be protected from the server. Ensuring user privacy is one of the keys to the ODBS model's success.
  - *Data privacy*: Users are not allowed to access more information than what they are querying on the server. In many situations, users must pay for what they have got from the server and the data owner does not allow them to obtain more than what they have paid for; moreover, even users do not want to pay for what they do not need because of the low bandwidth connections, limited memory/storage devices and so forth. This security objective is not easy to achieve and a cost-efficient solution to this issue is still an open question.
- *Query assurance*: Users are able to verify the correctness (authenticity and data integrity), completeness, and freshness of the result set. Among all security objectives, the query assurance is *always* appealed in the ODBS model. We succinctly explain these concepts as follows and more discussions can be found in (Mykletun & Tsudik, 2006; Narasimha & Tsudik, 2006; Dang, 2008; Li et. al., 2006; Pang et. al., 2005; Nguyen & Dang, 2008):
  - *Proof of correctness*: As a user queries outsourced data, it expects a set of tuples satisfying all query conditions and also needs assurance that data returned from the server originated from the data owner and have not been tampered with either by an outside attacker or by the server itself.
  - *Proof of completeness*: As a user queries outsourced data, completeness implies that the user can verify that the server returned *all* tuples matching all query conditions, i.e., the server did not omit any tuples satisfying the query conditions. Note that a server, which is either malicious or lazy, might not execute the query over the entire database and return no or only partial results. Ensuring the completeness of the query result aims to detect this unexpected behavior.
  - *Proof of freshness*: The user must be assured that the result set was generated with the most recent snapshot of the database. This issue must be addressed to facilitate *dynamic* outsourced databases, which frequently have updates on their data.
- *Secure auditing*: It is well-known that auditing and accountability play an indispensable role in e-business protocols and compliance (Jaquith, 2007; Natan, 2005). Even so, the way that the server can tackle auditing activities over encrypted databases is not a trivial matter despite recent advances in networking, data storage, and data processing. In particular, in the ODBS model, as privacy requirements are taken into account, auditing and accountability are much more difficult to achieve because their goals appear to be in contradiction (Burmester et. al., 2004).
- *Secure and efficient storage*: Although several commercial database vendors have already offered integrated solutions to provide data privacy within existing products, treating security and privacy issues as an afterthought often results in inefficient implementations (Iyer et al., 2004). Analyzing

issues in current storage technologies, looking at trade-offs between security and efficiency, and proposing new secure storage models will be challenging research topics of great interest in the ODBS/OXMLDBS model.

Figure 2. Security issues in outsourced (XML) databases



The above security requirements differ from the traditional database security ones (Castano et. al., 1995; Umar, 2004), and will in general influence the performance, usability and scalability of the ODBS/OXMLDBS model. Although there exist a vast number of research works on the above topics, to the best of our knowledge, none of them has *radically* addressed *security issues* in *outsourced XML data*. It is well-known that XML data have their own typical characteristics, which are very much different from data stored in traditional DBMSs (W3C, 2008; Thuraisingham, 2005). In particular, XML data are tree-structured data and it has been clearly proven in the literature that tree-indexed data have played an important role in both traditional and modern database applications (Dang, 2003). Therefore, security issues with outsourced tree-indexed data need to be resolved completely in order to materialize the ODBS model. Even so, this is not a trivial task, especially as tree-based index structures are outsourced to mistrusted servers. There are many challenging and open security issues in this case (Du & Atallah, 2000; Dang, 2005; Xie et. al., 2007; Nguyen et. al., 2007) (see Figure 2). The existing solutions are still considered immature. Section 3 will focus in depth on security issues in such XML databases, but in the outsourcing context.

Furthermore, note that the needs of the above-mentioned security requirements very much depend on particular scenarios in the ODBS model and this must be considered carefully. To identify such security enforcement needs for typical outsourcing scenarios, *a classification based on users' database access* and security requirements for each subclass are given below.

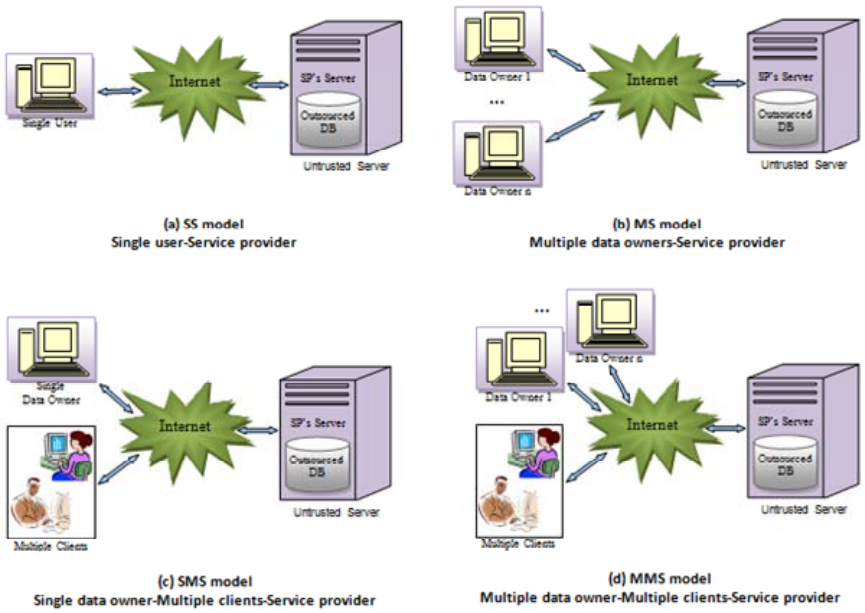
A user's remote database access-based classification. As presented in (Du & Atallah, 2000; Dang, 2006b; Mykletun et. al., 2004), there are a number of different ODBS models depending on users' database access *to the remote database*. In (Du & Atallah, 2000; Dang, 2006b), the authors distinguish four different e-commerce models that all require ensuring user privacy. Nevertheless, the two identified models are not in the context of the outsourced database because the data owner is himself the service provider. The other two are DC-UP (Data Confidentiality-User Privacy) and DC-UP-DP (Data Confidentiality-User Privacy-Data Privacy) models (Dang, 2006b). The DC-UP-DP model is exactly as illustrated in Figure 1b, where both the data owner and clients can query the outsourced database. On the contrary, in the DC-UP model, data owners are also unique clients. In this case, the data owner and clients as illustrated in Figure 1b are the same. In (Mykletun et. al., 2004), the authors distinguish between three outsourcing models: the first two, named *unified client* and *multi-querier* models, are the same as the DC-UP and DC-UP-DP ones, individually. A variant of the multi-querier model where an outsourced database is owned by a number of different owners is called the *multi-owner* model. We summarize all of these identified models, their variants and, based on system users' remote database access, classify them into *four* ODBS/OXMLDBS models *applicable in real-world application domains* as depicted in Figure 3. More details of these models are given below:

- *SS model* (**S**ingle user-**S**ervice provider, see Figure 3a): A single data owner is also the *unique* client, and its data is outsourced to the external server. We call both the data owner and client in this case *the user*. In this model, the data owner (also the client) is very much concerned about *data confidentiality* and *user privacy*, but *data privacy* is usually unimportant. This outsourcing model is similar to the *DC-UP* one. Again, *query assurances*, *secure auditing*, and *secure and efficient storage* are *all* required for *all* hosting-based outsourcing models.
- *MS model* (**M**ultiple data owner-**S**ervice provider, see Figure 3b): There are *multiple* data owners outsourcing data to the external server similarly to the SS model; however, each data owner is also the *unique* client. As a result, security requirements are the same as those of the SS model. The main differences between these two models are the way of accessing a remote database, and the techniques for ensuring security requirements (see section 2.2). Notably, the motivation for this model is rather straightforward. Consider the example of an outsourced insurance database. Each record in this database is created and maintained by a salesperson (of a particular branch) responsible for a particular customer. This salesperson then 'owns' the records that s/he creates.
- *SMS model* (**S**ingle data owner-**M**ultiple clients-**S**ervice provider, see Figure 3c): This is the most typical hosting-based outsourcing model where a single data owner outsources its database to the service provider. However, except for this data owner, multiple clients may also have access to the outsourced database on some agreement basis. This is among the most complex models in terms of security issues. The *data owner* is concerned about both *data confidentiality* and *data privacy* for both the external database server and its clients. The *client*, in turn, is apparently concerned about *user privacy* for both the *data owner* and *outsourcing server*. In some real-world cases, whose reasons may come from the data owner, service provider, or even the client itself (as discussed above), the client is also concerned about data privacy. Moreover, the data owner also takes the client role when accessing its outsourced data on the server and, in this case, the *data*

owner is concerned about *user privacy* as well. This outsourcing model is similar to the *DC-UP-DP* model.

- *MMS model* (**M**ultiple data owner-**M**ultiple clients-**S**ervice provider, see Figure 3d): This model is similar to the SMS one, except that there are n data owners owning the same outsourced database, $n \geq 2$, like the MS model. The challenging security issues for this model are the same as those for the SMS model. Besides, like the MS model, query processing and optimization are a big problem that emerges together with ensuring the security requirements. A trade-off between the two, that is, between security and performance, should be considered carefully (see section 2.2).

Figure 3. A user's database access-based classification for ODBS/OXMLDBS models.



For the sake of clarity, in Table 1 we summarize security requirements identified for the four ODBS/OXMLDBS models above. There is a special scenario in this table, the last row, that has not been discussed before and we will clarify it shortly in the next section.

Table 1. Security requirements in different ODBS/OXMLDBS models

No	ODBS/OXMLDBS models	Security requirements							
		Data confidentiality	Privacy concerns		Query assurances			Secure auditing	Secure and efficient storage
			User privacy	Data privacy	Correctness	Completeness	Freshness		
1	SS	√	√	-	√	√	√	√	√
2	MS	√	√	-	√	√	√	√	√
3	SMS	√	√	x	√	√	√	√	√
4	MMS	√	√	x	√	√	√	√	√
5	Multiple service providers (SP)	- (for each SP)	- (for each SP)	- (for each SP)	√	√	√	- (for each SP)	- (for each SP)

(√: fully required, -: not required, x: not fully required)

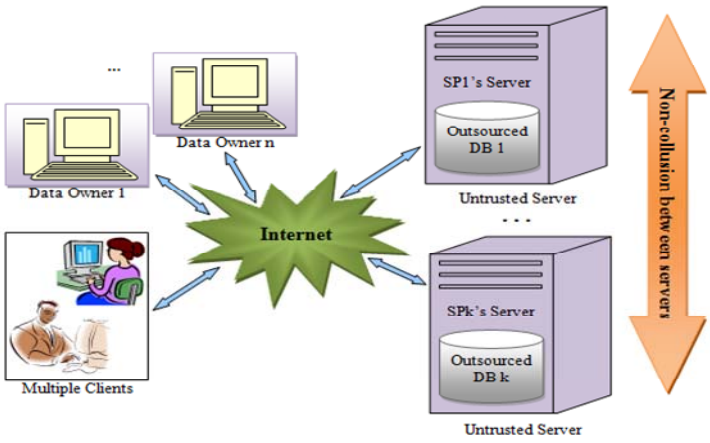
2.2. Existing Solutions to ODBS Models: An Overview

In Figure 2 we diagrammatically summarize security issues in the ODBS/OXMLDBS model, together with major references to the corresponding state-of-the-art solutions. An overview of these solutions is given below.

Ensuring Data Confidentiality

As shown in Figure 2, all crucial security objectives of the hosting-based outsourcing model have been investigated. To address the data confidentiality issue, most existing approaches opt to encrypt (outsourced) data before its being stored at the external server (Bouganim & Pucheral, 2002; Damiani et al., 2003; Dang, 2005; Evdokimov & Günther, 2007). Although an encryption scheme-based solution can protect the data from outsiders as well as the server, it introduces difficulties in the querying process, as it is hard to ensure the user and data privacy when performing queries over encrypted data. We will detail these issues later. In (Aggarwal et al., 2005), the authors argued that data encryption is unnecessary due to its negative impact on the system performance. Hence, they proposed an implementation architecture for all ODBS models where the data owner(s) outsources its data to several external database servers. Figure 4 illustrates this architecture in the MMS model.

Figure 4. Ensuring data confidentiality in the MMS model using multiple service providers.



The key idea in this implementation is to allow the client to partition its data across a number of logically independent database systems that *cannot communicate with each other*. The data partitioning is performed in such a way that it ensures that the exposure of the contents of any one database does not result in a violation of privacy objectives. The client executes queries by transmitting appropriate sub-queries to each database, and then gathering the results at the client side. Clearly, if the ODBS model is developed using this architecture, with *each service provider*, the data confidentiality, user and data privacy, secure auditing, and secure and efficient storage are unimportant (see Table 1). Nevertheless, user and data privacy would still be challenging problems in the context of the data owner and client communications concerning the *original query* and *final result*. The original query here means the query posed by a client before it is "split" into sub-queries, which will be sent to different servers. Similarly, the final result sent to the client is a combination of results of sub-queries. In addition, a severe drawback of

this outsourcing architecture is that it requires *non-collusion* between all service providers (see Figure 4), which is mostly impractical in the real-world. Thus, most existing and ongoing approaches still prefer some encryption scheme for the outsourced database confidentiality guarantee, and try to reduce related costs and complexities during the query processing.

Ensuring User and Data Privacy

In general, in order to address the issue of user and data privacy, outsourced data structures employed to manage the data storage and retrieval should be considered. Notably, the problem of user privacy has been quite well solved, even without special hardware (Smith & Safford, 2001), if the outsourced database contains only encrypted records and no tree-based indexes are used (see (Dang, 2006b) for an overview). Conversely, the research result is less incentive if such trees are employed, although some proposals have been made recently (Lin & Candan, 2004a; Dang, 2006b). In our previous work (Dang, 2006b), we proposed an extreme protocol for the ODBS model based on private information retrieval (PIR)-like protocols (Asonov, 2001). It would, however, become prohibitively expensive if only one server were used to host the outsourced data (Chor et. al., 1995; Asonov, 2001). Damiani et al. (2003) also gave a solution to query outsourced data indexed by B+-trees, but their approach does not provide an obvious way to traverse the tree, and this may compromise security objectives (Lin & Candan, 2004a; Dang, 2006a). Lin and Candan (2004a; 2004b) introduced a computational complexity approach to solve the problem with sound experimental results reported over XML datasets. Their solution, however, supports only an obvious search on outsourced search trees, but not any insertion, deletion or modification. Hence, their solution cannot be applied to dynamic outsourced search trees where several items may be inserted and/or removed, or existing data can be modified. In our recent work (Dang, 2005; Dang, 2006a), we analyzed and introduced techniques to completely solve the problem of data confidentiality and user privacy, but query assurance, in the ODBS model with dynamic tree-indexed data supports. In (Dang & Son, 2006) (also Dang, 2008) we extended the previous work to solve all the three security objectives of query assurance as discussed in section 2.1. In particular, the extended approach can be applied to all kinds of search trees, including multidimensional index structures that have played an important role in both traditional and modern database application domains (Dang, 2003).

Contrary to user privacy, although there are initial research activities (Gertner et. al., 1998; Du & Atallah, 2000; Boneh et. al., 2004; Dang, 2006b; Evdokimov & Günther, 2007), the problem of data privacy still needs much more attention. Gertner et al. (1998) first considered the data privacy issue in the context of PIR-like protocols and proposed the symmetrical PIR (SPIR) protocol to prevent users from knowing anything more than the answers to their queries. Unfortunately, such PIR-based approaches cannot be applied to the ODBS model because the data owners in PIR-like protocols are themselves the database service providers. Du and Atallah (2000) introduced protocols for secure remote database access with approximate matching of four different ODBS models requiring different security objectives among those presented in the previous section. Even so, their work did not support outsourced tree-indexed data. In our recent work (Dang, 2006b), we presented a solution to ensure data privacy in the ODBS model that can also be applied to tree-indexed data. Nevertheless, our proposed solution must resort to a trusted third party, which is currently uneasy to find in practice. Of late, Evdokimov and Günther (2007) proposed a new data privacy-enabling ODBS scheme. This newly proposed scheme can avoid a problem of many previous solutions: erroneous tuples that do not satisfy the select condition may be returned to the client in the result set but the probability of such an error is negligible. This capability improves the performance and simplifies the development process of a client's software. Even then, though with a small probability, data privacy is theoretically still prone to be erroneous. The only introduced scheme

that allows performing search on encrypted data and does not require post-filtering is described in (Boneh et. al., 2004). This scheme, however, can hardly be applied to most ODBS models since searching an encrypted database is restricted to the search with predefined keywords. Importantly, neither of these approaches takes indexed data into account radically. Actually, a crucially challenging problem related to security in all models is to provide data privacy. This requirement is still open to the research community due to its interdisciplinary complexities.

Ensuring Query Assurance

Recently, addressing the three issues of query assurance has attracted many researchers and, as a result, a number of solutions have been proposed (e.g., Boneh et. al., 2007; Mykletun et. al., 2004; Pang & Tan, 2004; Pang et. al., 2005; Narasimha & Tsudik, 2006; Sion, 2005; Dang & Son, 2006; Li et. al., 2006). However, except for (Dang & Son, 2006; Li et. al., 2006), none of the previous work has given a solution to the problem of guaranteeing the query result freshness. To prove the correctness of a user's query results, the state-of-the-art approaches (Boneh et. al., 2007; Mykletun et. al., 2004; Pang & Tan, 2004; Sion, 2005) employed some aggregated/condensed digital signature scheme to reduce the communication and computation costs. First, Boneh et al. (2003) introduced an aggregated signature scheme that allows aggregation of multiple signers' signatures generated from different messages into one short signature based on elliptic curves and bilinear mappings. Despite the big advantage that this scheme can be applied to different ODBS models, it must bear a disadvantage related to the performance. As shown in (Mykletun et. al., 2004), the computational complexity of Boneh et al.'s (2003) scheme is quite high for practical uses in many cases. Second, Mykletun et al. (2004) introduced an RSA-based condensed digital signature scheme that can be used for providing the proof of correctness in the ODBS model. Their scheme is concisely summarized as follows.

Condensed-RSA digital signature scheme. Suppose pk=(n,e) and sk=(n,d) are the public and private keys, respectively, of the RSA signature scheme, where n is a k-bit modulus formed as the product of two k/2-bit primes p and q. Assume $\varphi(n)$=(p-1)(q-1), both public and private exponents e, d $\in Z_n^*$ and must satisfy ed≡1 mod $\varphi(n)$. Given t different messages $\{m_1, ..., m_t\}$ and their corresponding signatures $\{s_1, ..., s_t\}$ that are generated by the *same* signer, a condensed-RSA signature is computed as follows: $s_{1,t}=\prod_1^t s_i$ mod n. This signature is of the same size as a single standard RSA signature. To verify the correctness of t received messages, the user must multiply the hashes of all t messages and check that $(s_{1,t})^e \equiv \prod_1^t h(m_i)$ mod n.

The above scheme is possible because RSA is multiplicatively homomorphic. However, this scheme is applicable only for a single signer's signatures. In (Dang & Son, 2006; Dang, 2008) we applied this scheme to the SS model to provide correctness guarantees of the received tree nodes from the server (see section 3). Sion (2005) also employed it to address the correctness of query results in his proposed scheme for the SS model. Besides, Pang and Tan (2004) applied and modified the idea of Merkle Hash Trees (MHT) (Merkle, 1980) to provide a proof of correctness for edge computing applications, where a trusted central server outsources parts of the database to proxy servers located at the edge of the network. However, the proposed approach does not check for completeness of query results. More seriously, Narasimha and Tsudik (2006) pointed out possible security flaws in this approach.

Furthermore, there are a number of approaches to deal with the completeness of a user's query results (Sion, 2005; Pang et al., 2005; Narasimha & Tsudik, 2006; Dang & Son, 2006). First, Sion (2005)

proposed a solution to provide such assurances for arbitrary queries in outsourced databases. This solution is built around a mechanism of runtime query "proofs" in a challenge-response protocol. Concretely, the data owner partitions its data into k segments {S1, ..., Sk}, computes hashes for each segment, H(Si), i= , then stores them all together at the service provider. In addition, the data owner also calculates some "challenge tokens" for Si. Actually, the challenge tokens are queries that the data owner already knows their results, which can be used for verification later. Whenever a batch of queries is sent to the server, certain challenge token(s) are also sent together. The result set is then verified using the challenge tokens for its completeness. Although this approach can be applied to different query types, query completeness cannot be guaranteed 100% because there is the chance that a malicious server can "guess" and return the correct answer to the challenge token together with fake result sets for other queries in the batch. Moreover, this approach is cost-inefficient for database updates because the challenging answers must be recalculated. Seriously, although the author did not aim to address the user privacy issue, we should note that user privacy in this approach may be compromised because the server knows what data segments are required by the user in order for inference and linking attacks to be be conducted (Dang, 2006b; Damiani et al., 2003). Second, Pang et al. (2005) introduced a solution based on aggregated signature schemes and MHT. It is an extension of their previous work (Pang & Tan, 2004), which has been proven insecure due to some possible security flaws (Narasimha & Tsudik, 2006). Last, Narasimha and Tsudik (2006) developed an approach, DSAC–digital signature aggregation and chaining, that achieves both correctness and completeness of query replies. However, in their approach, tuples must be pre-sorted in ascending order for each searchable dimension for calculation of the signature chain, and thus it does not support outsourced tree-indexed data because the order of the contents of tree nodes is undetermined. This requirement also has a tremendous negative impact on data updates, causing degeneration of the system's total performance. Recently, in (Dang & Son, 2006; Pang et. al., 2005) the authors presented comprehensive solutions to address all the three aspects of query assurance in the SS model. These approaches are considered as the vanguard ones to address dynamic outsourced databases. Notably, Dang and Son's (2006) presented approach, which is based on access redundancy and node swapping techniques (Lin & Candan, 2004a; 2004b), also supports outsourced multidimensional index structures.

Ensuring Secure Auditing

Furthermore, assume that we have a running system with the above security objectives being satisfied, then the question "How can the server conduct auditing activities in systems provided with such security guarantees?" is still left open to date. Due to the user privacy (see section 2.1), the server may not know who is accessing the system (e.g., Lin & Candan, 2004a; Dang, 2006b; Dang, 2008), what they are asking for, what the system returns to the user, and thus how it can effectively and efficiently tackle the accountability or develop intrusion detection/prevention systems. The goals of privacy-preserving and accountability appear to be in contradiction, and an efficient solution to balance the two is still open. There is just some research work on this topic. Typically, in (Burmester et. al., 2004) the authors, among the trailblazers, did investigate the problem of the conflicts existing between secure auditing/accountability and privacy in a variety of application domains. Research directions for balancing the needs of both were also highlighted. In the past, although there are some hardware-based approaches to solving security issues in ODBS models such as (Smith & Safford, 2001; Mykletun & Tsudik, 2005) and the secure auditing problem is also of their concerns no practically proven solution has been given. In section 4, we will further discuss these hardware-based approaches.

Ensuring Secure and Efficient Storage

As we can observe in Figure 2, there exists another interesting question: Is there a secure and efficient storage model for the outsourced database? However, all existing approaches for dealing with the concerned security problems do not make any assumption about the DBMS and its storage model. It is simply understood that existing commercial DBMS can be employed with some additional settings for the stored data at the server site, while the underlying storage model remains unchanged (Hacigümüs et. al., 2002; Bertino et. al., 2004; Damiani et al., 2003; Dang, 2008; etc.). In a recent research (Iyer et al., 2004), the authors carried out insightful studies on this problem, and proposed a framework for efficient storage security in RDBMS. Their approach is motivated by crucial weaknesses of today's database security solutions, namely poor efficiency, inflexibility, and non-encryption of indexes. The poor efficiency appears because the underlying model used by DBMS introduces significant computational overheads. A new solution can be possible only if the means of storing records on the disk blocks is modified. The inflexibility is manifested by the wrong encryption granularity, leading to the fact that even non-sensitive data is also encrypted in many cases. Lastly, most existing solutions offered by commercial DBMS to data encryption problem do not allow indexing techniques or must sacrifice some important functionalities (e.g., most encryption schemes are not order-preserving, thereby losing range query functionality). More concretely, Iyer et al. (2004) introduced a secure storage model and a key management architecture that enable efficient cryptographic operations while trying to maintain an acceptable level of security. The proposed approach is based on grouping sensitive data and non-encryption of non-sensitive data to minimize the number of encryption operations. Although some preliminarily sound experimental results have been reported, this model is only the first step towards a sustainable, secure and efficient storage model for ODBS/OXMLDBS models because it still assumes that the server is trusted, which typically does not exist in ODBS/OXMLDBS models.

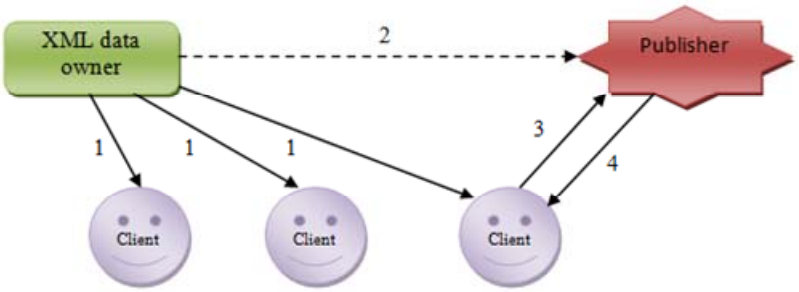
Until now, it has been clear that dealing with security issues in ODBS models encounters many difficulties if *tree-structured data* is outsourced to untrusted servers (Du & Atallah, 2000; Dang, 2006b). On the other hand, as discussed in the literature, tree-based index structures play an indispensable role in both traditional and modern database applications (Dang, 2003). Unfortunately, XML data is tree-structured inherently. Most of approaches to security issues in outsourced XML databases were inspired by, and originated from, the findings of solutions to the same issues in ODBS models with traditional numeric and text databases. In the next section, we will present and discuss the state-of-the-art approaches proposed in the most recent publications to *securely* managing the storage and retrieval of outsourced tree-indexed/XML data.

3. STATE-OF-THE-ART APPROACHES TO SECURITY ISSUES IN OXMLDBS MODELS

3.1. Authentic Publication of XML Documents: A Semi-OXMLDBS Model

At the very beginning, in (Devanbu et al., 2001) the authors proposed an approach for signing XML documents allowing untrusted servers to answer certain types of path and selection queries. The main research idea in this work and other similar ones is very similar to that of the SMS model (see Figure 3c). Figure 5 depicts this working model.

Figure 5. Authentic publication of XML documents: Document is created by owner, who uses a one-way hash function to digest it. Clients receive the digest through an authentic channel, e.g., using a public-key signature (1) and the document itself is sent to a publisher (2). In response to a query from a client (3) the publisher returns both an answer and a verification object using the MHTs and digital signatures, certifying the correctness and completeness of the answer (4).



In this model, the XML data owner and clients play the same role as those in the SMS model. The publisher in the authentic publication of XML documents, however, is slightly different from the service provider in the SMS model: this publisher may not store the owner's encrypted data, but it can keep only the owner's plaintext XML data. Although Devanbu et al.'s (2001) solution, which is based on the MHT and a newly proposed data structure (xtrie), to provide correctness and completeness of the query results, overcomes several limitations of previous work (e.g., only one digital signature over an XML document is needed to certify answers to arbitrary selection queries over such documents, providing both correctness and completeness of answers, etc.), not all privacy-related issues are considered (i.e., user and data privacy). Therefore, this working model is not considered as an OXMLDBS model, but a semi-OXMLDBS model. Moreover, the auxiliary structure, the xtrie, is based on the DTD, but the proposed solution works best only on non-recursive DTDs. Even then, the empirical analysis indicated that a majority of published DTDs are non-recursive, and thus this approach will be quite useful in a variety of contexts. In general, the proposed techniques for ensuring query assurance objectives in Devanbu et al.'s (2001) solution can be further explored for possible application to OXMLDBS models where a server truly stores and manages a data owner's encrypted data.

Very similarly to the above approach, in (Bertino et. al., 2004) the authors carried out extensive research in secure XML document distribution and provided a comprehensive architecture for selective, authentic, and complete third-party publication of XML documents. In the same line as Devanbu et al.'s (2001) solution, this approach also considered the publisher to be untrusted for the requirements of correctness and completeness. MHTs, digital signature schemes, and credential-based techniques have been employed to provide correctness as well as completeness proofs for the query answers. Only one thing would be considered as being new in Devanbu et al.'s (2001) solution: that this introduced comprehensive architecture also incorporates a policy configuration that is technically a certificate issued and signed by the XML data owner containing information about access control policies that apply to the user. By utilizing this, the publisher can determine and filter out (certain portions of) XML documents before returning the query result to the client. Once again, none of these approaches is for encrypted XML databases and, consequently, data confidentiality and privacy-related issues for the publisher are not assured.

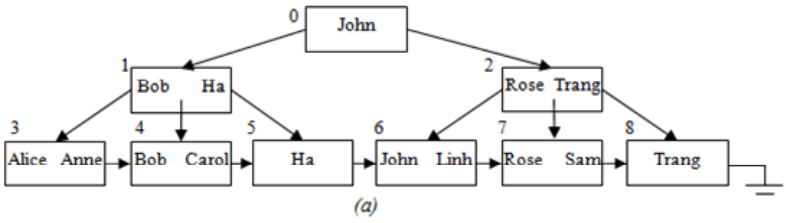
Recently, in (Bertino et. al., 2007), the authors further developed the above ideas and introduced a push-based system for distributing information through the Internet. Such systems are today becoming

increasingly popular and widely used. The widespread use of such systems raises significant security concerns like those in OXMLDBS models, such as confidentiality, integrity and authenticity, and completeness of the distributed data. To cope with such issues, Bertino et al. (2007) described a system for securing push distribution of XML documents, which adopts digital signature and encryption techniques to ensure the above-mentioned security requirements and allows the specification of both signature and access control policies. The authors also described the implementation of the proposed system and presented an extensive performance evaluation of its main components. Although all privacy-related issues were still not addressed, this is a good reference for further investigations into such semi-OXMLDBS models and security concerns.

3.2. Oblivious Operations on Outsourced Search Tree

Basic settings. As we know, XML documents have tree-like structures (W3C, 2008; Lin & Candan, 2004a; Brinkman et. al., 2004) and XML has become a de facto standard for data exchange and representation over the Internet. It is also well-known that basic operations of tree-based index structures include search and updates (modify, insert, delete). To facilitate security requirements in OXMLDBS models (see section 2.1), techniques allow users to operate on their outsourced tree-structured data on untrusted servers without revealing information about the query, and result, and outsourced data itself must be studied.

Figure 6. An example of B+-tree (a) and the corresponding plaintext and encrypted table (b).



(a)

B⁺Table

NID	Node
0	(1,John,2,-,-1)
1	(3,Bob,4,Ha,5)
2	(6,Rose,7,Trang,8)
3	(Alice,Anne,4)
4	(Bob,Carol,5)
5	(Ha,-,6)
6	(John,Linh,7)
7	(Rose,Sam,8)
8	(Trang,-,-1)

B⁺EncryptedTable

NID	EncryptedNode
0	Aza1n2g3Kh75nhs&lkgf
1	K9&8ra§ÖÄajh³q91µαβ√
2	Iv&$uye"µnÜis57ß@©±
3	L?{inh*ß²³&§gnaDkhanh
4	Owh09a/[%?Ö#Aj2ktran
5	jVHß}[aHo$§anguGdang
6	#Xyi29?ß~R@€>Khxml
7	$B³¦jKDÖbd0K3}%§cha
8	T-§µran&gU19=75mter1

(b)

Although tree-based index structures have proven their advantages over both traditional and modern database applications, they introduce numerous research challenges as database services are outsourced to untrusted servers (Du & Atallah, 2000; Dang, 2006b). To detail the problem, Figure 6a illustrates an example of B+-tree for an attribute CustomerName with sample values. Assume a user is querying all customers whose name is Ha on this tree. If we do not have a secure mechanism for tree storage and query processing, a sequence of queries that will access in sequence nodes 0, 1, and 5 for the above query will be revealed to the server. In addition, the server also realizes that the user was accessing nodes 0, 1, and 5, which are the root, an internal node, and a leaf node, respectively, of the tree, and so the user privacy is compromised. More seriously, using such information collected gradually the server can

rebuild the whole tree structure and infer sensitive information from the encrypted database. Besides, the user will also receive information showing that there are at least two other customers named John and Bob in the database.

To cope with these issues, some settings must be prepared to facilitate oblivious operations on the tree. Damiani et al. (2003) showed that encrypting each tree node as a whole to ensure the data confidentiality is preferable because protecting a tree-based index by encrypting each of its fields would disclose to the untrusted server the ordering relationship between the index values. Besides, each node is identified by a unique *node identifier* (NID) and the unit of storage and access is a tree node. The original tree is then stored at the server as a table with two attributes: *NID* and *an encrypted value* representing the node content. In our running example, Figure 6b shows the corresponding plaintext and encrypted table used to store the B^+-tree at the server over schema $B^+EncryptedTable = \{NID, EncryptedNode\}$.

Oblivious basic operations on outsourced XML data trees. Based on the above settings, there are some proposed approaches to facilitate basic operations on the outsourced search tree. Firstly, Damiani et al. (2003) analyzed potential inference and linking attacks, and proposed a hash-based indexing method suitable for exact match queries. To process range queries, they proposed a solution employing B+-trees with the above settings. However, Dang (2006b) indicated that this solution does not provide an oblivious way to traverse the tree and this can be exploited by the untrusted server to carry out inference and linking attacks. Moreover, it also does not support outsourced multidimensional index structures. Later, Lin and Candan (2004a) developed an algorithm based on access redundancy and node swapping techniques to provide oblivious search on outsourced tree-indexed data.

Access redundancy and node swapping: Each time a user requests access to a node from the server, it asks for a redundancy set of m nodes consisting of at least one empty node along with the target one. The user then: (1) decrypts the target node, (2) manipulates its data, (3) swaps it with the empty node, and (4) re-encrypts all m nodes and writes them back to the server. Note that, to prevent the server from differentiating between read and write operations, a read operation is always followed by a write operation. This requires re-encryption of nodes using a different encryption scheme before they are rewritten to the server. With this technique, the possible position of the target node is randomly distributed over the data storage space at the untrusted server, and thus the probability that the server can guess the target node is 1/m.

To realize oblivious tree traversal, securely managing the root node address and empty node lists is also important: A special node called SNODE whose NID and decryption key are known to all valid users keeps pointers ROOTS pointing to the roots of all outsourced search trees; and empty nodes are stored in hidden linked lists. Based on these basic settings, Lin and Candan's (2004a) algorithm is presented as follows:

Algorithm 1: *Oblivious Tree Traversal/Search*

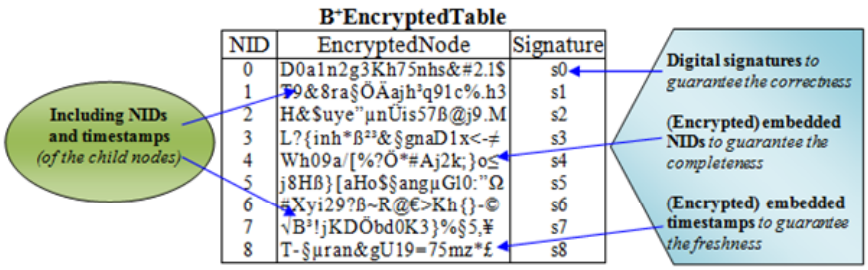
1. Lock and fetch the SNODE, let it be PARENT. Find the root and let it be CURRENT.
2. Select a redundancy set for the CURRENT, lock nodes in the set, and let the empty node in the set be EMPTY.
3. Update the PARENT's pointer to refer to the EMPTY, and release locks on the PARENT level.
4. Swap the CURRENT with the EMPTY.
5. If the CURRENT contains the needed data, return CURRENT. Otherwise:
6. Let the CURRENT be PARENT, find the child node to be traversed next, let it be CURRENT, and repeat steps 2 through 5.

Note that the original approach above works only with uniformly distributed data sets. In (Lin & Candan, 2004b), the authors enhanced this approach so that it can also work with non-uniformly

distributed datasets, i.e., from the server's view, node accesses are always uniformly distributed. Furthermore, although the above approach supports only *oblivious* tree search operations, the two employed techniques have served as the basis for our further investigation. In (Dang, 2005; Dang, 2006a) we identified critical issues and limitations of Lin and Candan's approach and developed pragmatic algorithms for privacy-preserving search, insert, delete, and modify operations that can be applied to a variety of *dynamic* outsourced tree-based index structures and the *SS* model. Despite our previous work providing the vanguard solutions for this problem with sound empirical results, it did not consider any of the query assurance aspects.

Query assurances for outsourced tree-indexed/XML data. In (Dang & Son, 2006; Dang, 2008) we extended our previous work and presented a full-fledged solution to the problem of ensuring the *correctness, completeness,* and *freshness* for all basic operations (insert, delete, modify, point, and range queries) on *dynamic* XML/outsourced tree-indexed data. Experimental results with real *multidimensional* datasets (R-Tree Portal, 2006) and the kd-tree have confirmed the efficiency of our proposed solution. Figure 7 shows *core settings* in our extended approach.

Figure 7. Settings for verifying correctness, completeness, and freshness guarantees.



Firstly, the data owner computes and signs the hash *h(m)* of each encrypted node *m*. Next, it stores the signature together with EncryptedTable at the server. The table schema stored at the server now becomes *EncryptedTable = {NID, EncryptedNode, Signature}* (see Figure 7). With this setting, users can then verify each returned node using the data owner public key, thereby ensuring *the correctness* of the result set. To minimize the costs, we adopted the condensed-RSA digital signature scheme (see section 2.2) for nodes in the redundancy set. Secondly, in order to guarantee the query completeness, in our context, as a user asks the server for a redundancy set A of t nodes $A=\{m_1, ..., m_t\}$ and the server returns him a set R of t nodes $R=\{n_1, ..., n_t\}$, the user must be able to verify that $A=R$. As presented above, a user asks for any encrypted nodes through their NIDs. Therefore, the user should be provided with a means of verifying that NID of each m_i, $i=\overline{1,t}$, equals NID of each corresponding n_i, $i=\overline{1,t}$. To ensure this, our solution is embarrassingly simple: an NID is encrypted with the corresponding node contents and this encrypted value is stored at the server side, together with its signature as described above. Users can then check if the server returned the required NIDs (*the completeness*) and verify the query result correctness all together. Lastly, in order to address the freshness issue, users must be able to verify if the server returned the most up-to-date required tree nodes. Our solution is uncomplicated but sound and complete: A timestamp of each child node is stored at its parent node. This timestamp changes *only* as the child node *contents* (but not its address) are updated. A user can then check (from the root) if the server returned the latest version of the required node (*the freshness*). Note that, SNODE should keep the root's timestamp in addition to other information as discussed previously, and each qualified user must be informed about

SNODE's timestamp (e.g., by the data owner who made the changes to the root's contents). Remarkably, to the best of our knowledge, none of the previous work has addressed all the above three security issues of query assurance in the ODBS/OXMLDBS model with regards to *dynamic multidimensional outsourced trees*. Our work therefore provides a vanguard solution for this problem.

In addition, since XML data has tree-like structures and queries can be expressed as traversal paths on these trees, the solution introduced above can be utilized for secure outsourcing of XML documents. Compared with existing PIR techniques (Asonov, 2001), this proposed solution does not need database replication and requires less communication, and is thus practical (Lin & Candan, 2004b; Dang, 2006a). However, because these solutions have been developed only for the SS model, they are not directly applicable to other OXMLDBS models due to their different security requirements.

3.3. Secure Multi-Party Computation-Based Approaches

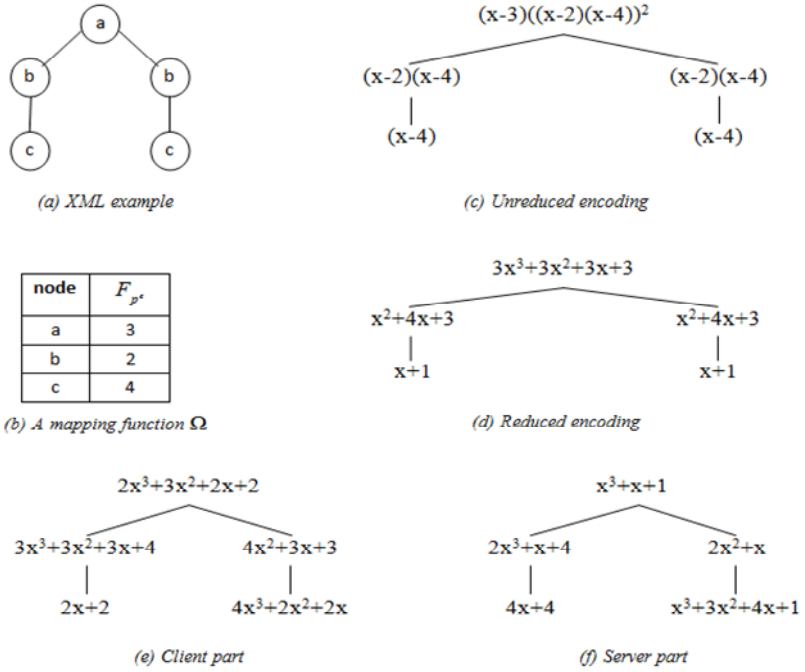
In a multi-party computation (MPC) we have a given number of participants p1, p2, ..., pN, each having a private data, respectively d1, d2, ..., dN. The participants want to compute the value of a function F on N variables at the point (d1, d2, ..., dN). An MPC protocol is said to be secure if no participant can learn more from the description of the public function and the result of the global calculation than what s/he can learn from his/her own entry under particular conditions depending on the model used. In (Brinkman et. al., 2004), the authors, inspired by secure MPC, introduced a new approach to query outsourced XML databases. First, a plaintext XML document is transformed into an encrypted database as follows:

1. Define a mapping function $\Omega : node \rightarrow F_{p^e}$, which maps the nodes' tag names to values of the finite field F_{p^e}, where p^e is a prime power (p is a prime and e is a positive integer) which is larger than the total number of different tag names (see Figure 8b). Although Ω may be chosen arbitrarily, the client should keep it secret to prevent the server from seeing the query.
2. Transform the tree of tag names (see Figure 8a) into a tree of polynomials (see Figure 8c and Figure 8d) of the same structure where each node is transformed to *f(node)* where function *f: node* $\rightarrow F_{p^e}[x]/(x^{p^{-1}}-1)$ is defined recursively: $f(node)=x-\Omega(node)$ if *node* is a leaf node, or $(x-\Omega(node))\prod_{d \in child(node)} f(d)$, otherwise. Here *child*(*node*) returns all children of a *node*. To avoid large degree polynomials a finite ring is chosen. Two different rings have been investigated: $F_q[x]/(x^{q-1}-1)$ (where q is a prime power $q=p^e$) and $Z[x]/(r(x))$ (where $r(x)$ is an irreducible polynomial). In the first case, the coefficients of the polynomials are reduced modulo q. With $Z[x]/(r(x))$, the polynomial is reduced modulo an irreducible polynomial $r(x)$. The running example as shown in Figure 8 is according to the first finite ring.
3. Split the resulting tree into a client (see Figure 8e) and a server tree (see Figure 8f). Both trees have the same structure as the original one. The polynomials in the client tree are generated by a pseudorandom generator. The polynomials of the server tree are chosen such that the sum of a client node and the corresponding server node equals the original polynomial.
4. Since the client tree is generated by a pseudorandom generator, it is sufficient to store the seed on the client. The client tree is discarded and, when necessary, it can be regenerated using the pseudorandom generator and the seed value.

Next, in the querying phase, it is simple to check whether a node N is stored somewhere in a subtree by evaluating the polynomials of both the server and the client at Ω(N). If the sum of these evaluations

equals zero, this means that N can be found somewhere in the subtree N. Note that everything is calculated modulo q. To find out whether N is the root of this subtree, we have to divide the unshared polynomial by the product of all its direct children. The result will be a monomial (x−t) where t is the mapped value of the node.

Figure 8. An example for Brinkman et al.'s (2004) approach based on secure MPC.



The above approach, however, is only for storing and retrieving trees of tag names, but not the actual data between the tags. Later, Brinkman et al. (2005) presented an extension of their previous approach to address this problem. They proposed a representation of XML documents allowing for searching in data nodes by simply transforming all data nodes to their trie representation (Fredkin, 1960). Having translated the original XML tree into a trie, the same presented strategy can be employed to encode the document and carry out the search. Even now, clients still have to store a sheer volume of XML meta-data and tackle heavily computational tasks as does the service provider. Next, because the tree structure transformed from the XML document is in plaintext at the server, reference attacks may be launched to compromise data confidentiality and user privacy. In addition, data privacy has not been discussed because the proposed approach is applicable only to the SS model. More seriously, query assurances have not been investigated and they are difficult to address efficiently because no efficient multi-purpose schemes for secure MPC are known to us at the moment. Finally, dynamic outsourced XML documents have also not been studied in this approach.

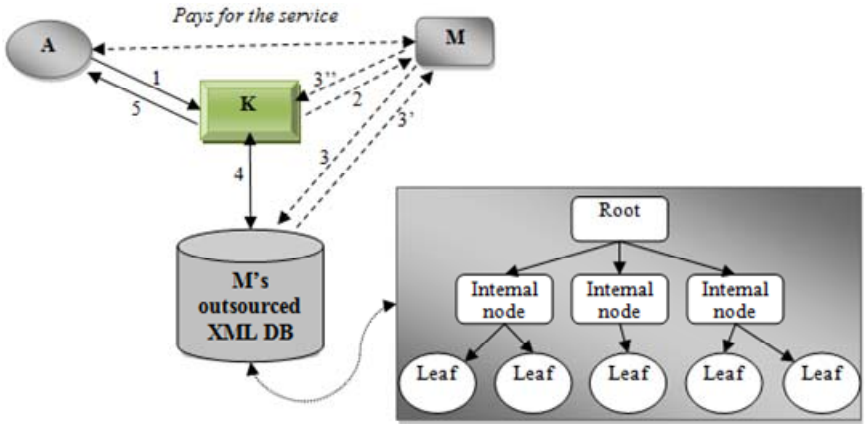
3.4 Trusted Third Party-Based Approaches

As far as we know, there is no solution to the SMS/MMS models for outsourced tree-structured data for all concerned security objectives. Relying on an assumption that quite solid solutions have been developed for the SS/MS models, Dang (2006b) proposed a protocol to ensure all security requirements

of the SMS/MMS models resorting to a trusted third party (TTP), namely K. The use of a TTP is to change this outsourcing model, which is very hard to address directly, to a better solved SS model. We describe five basic steps to process a query Q sent from a client A as follows (see Figure 9):

1. Client *A* sends *Q* to *K* for querying the data owner *M*'s outsourced XML database *DB*.
2. When receiving *Q*, *K* informs *M* (for billing, for example) and waits for approval from *M* in order to access *DB*.
3. On receiving *A*'s access request from *K*, *M* informs *DB* so that *K* can query *DB* on behalf of *M*. After receiving *DB*'s acknowledgement, *M* informs *K*.
4. From this time, *K* takes *M*'s role in the SS model as discussed in section 2, and it can access *DB* using any security protocols designed for the SS model (note that *A* is only capable of retrieving information from *DB*, but not updating *M*'s outsourced XML data in *DB*).
5. Finally, *K* filters and returns to *A* only the results of *Q*. Obviously, *K* has been informed by *M* what *A* is able to get from the database, but *M* will not be informed what *A* has got regarding any *Qs* to ensure *A*'s privacy wrt. *M*.

Figure 9. A secure protocol for trusted third party-based approaches.



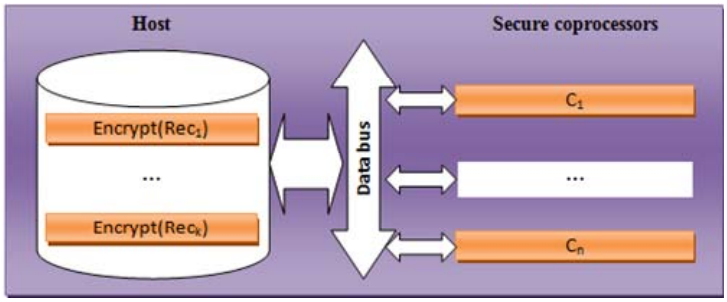
In practice, a TTP is difficult to find and it is also the only weakness of this protocol. With the assumption that we can establish such a TTP, it is easy to prove that the above protocol ensures all security objectives for this SMS/MMS models, i.e. data confidentiality, user and data privacy, as well as query assurances. Eliminating K from this protocol, while still ensuring all security objectives for this ODBS/OXMLDBS model, is an open research question. In the same line as our approach but relying on special hardware equipment, IBM has developed secure co-processors acting like K (IBM, 2008b; Smith & Safford, 2001). We will elaborate on this hardware-based approach in the next section.

The above approach is a general idea to address security issues in the SMS/MMS models and it can be applied to not only outsourced XML databases, but also any general tree-indexed data outsourced. Nevertheless, even though special hardware is employed, a number of challenging problems related to improving the system performance must be taken into account (Dang, 2006b; Mykletun & Tsudik, 2005). This presents new significant research problems of great importance in the future.

3.5 Hardware-Based Approaches

Contrary to the past claims that special-purpose computers for security requirements like those of OXMLDBS models are infeasible but trivial (Goldreich & Ostrovsky, 1996), the hardware-based approach is no longer infeasible. Secure coprocessor (SC) research has advanced the state-of-the-art (IBM, 2008b), and now permits any researcher at a reasonable cost to build a private information server by taking a host machine with ample PCI slots and inserting these coprocessors. In (Smith & Safford, 2001), the authors presented an IBM secure coprocessor-based server architecture that can efficiently process a user's queries without violating security and privacy objectives, such as data confidentiality, user and data privacy. Figure 10 depicts this server architecture. This system consists of a single server that has a number of SCs, and that provides a query service to a database containing records. The stored records are encrypted and authenticated. Actually, the SCs act like the TTP as described in Figure 9, and thus this architecture could be considered as a practical implementation of the general idea presented in Dang (2006b). With the support of such special hardware, it is clear that the new server is partly capable of satisfying security objectives in OXMLDBS models.

Figure 10. A server architecture based on secure coprocessors.



Recently, some hardware-based solutions to security issues in OXMLDBS models have been proposed (Bouganim & Pucheral, 2002; Mykletun & Tsudik, 2005), employing IBM-like SCs, e.g., IBM 47xx SC family (IBM, 2008b). More technically, a SC is a general-purpose computer that can be trusted to carry out its computations unmolested, even if an adversary has direct physical access to the device. It is equipped with a processor, non-volatile secure memory, input devices, back-up battery and is fully enclosed in a tamper-proof container that cannot be opened without triggering sensors that alert that an attack is taking place. In the case that a penetration of the device is detected through the signaling by sensors that are monitoring possible attack venues, an alarm is triggered and all contents in the secure memory are erased, possibly by destroying the physical memory chip by leaking acid onto it (IBM, 2008b; Mykletun & Tsudik, 2005). Notwithstanding, SCs are still far away from meeting the functionalities expected by users due to the hardware-related technologies (see section 4), and thus they still do not completely solve the security issues of concern.

3.6 Hybrid Tree-Based Approaches

Although there exist approaches employing hybrid tree-based structures to address security issues in ODBS model like (Li et. al., 2006), they unfortunately have not been studied for OXMLDBS models. Very recently, in (Nguyen et. al., 2007; Nguyen & Dang, 2008), on the basis of MHTs and B+-tree, the

authors introduced a new indexing structure for outsourced XML data to solve the data confidentiality and query assurances in OXMLDBS models. Even now, their approach can be extended by employing techniques presented in section 3.2 to deal with the privacy-related issues. Below, we introduce Nguyen and Dang's (2008) original approach.

Node-based XML document transformation. Each XML document has a schema tree that defines the relation between nodes and their attributes. A schema tree consists of two node types: element node (t-node) and attribute node (a-node). t-nodes and a-nodes are stored in a single table and each node has enough data to reconstruct the original XML text:

t-node(nodeid, xtype, datatype, nameID, pnodeID, lmaID, value)
a-node(nodeid, xtype, datatype, nameID, pnodeID, sibID, value)

where, xtype is used to distinguish t-node and a-node; datatype determines type of the data value; nameID is the node's identifier; pnodeID refers to parent node's tuple; lmaID refers to the left-most attribute of the node; sibID refers to the right-sibling attribute; value is the value of the node/attribute. For data confidentiality, each record is serialized into an encrypted binary string before outsourcing.

Nested Merkle B+-Tree. An important factor for a feasible solution to query assurance of outsourced XML databases depends on index structures. By embedding extra information into this structure, we could achieve query assurances. MHTs and digital signature schemes, as discussed in (Li et. al., 2006; Nguyen & Dang, 2008), are typically employed to provide the proof of query correctness. To obtain an effective proof for query completeness, as shown in (Narasimha & Tsudik, 2006; Nguyen & Dang, 2008), all elements must be sorted by two criteria: (path, value) and (path, parent, value), where path is the path from the tree's root to a given node. However, no existing data structures could help this. Nguyen and Dang (2008) introduced a novel Nested Merkle B+-Tree to facilitate the storage/retrieval and ensure the query assurance for outsourced XML databases.

First, all possible paths from root to leaves are listed out, and each path is associated with a unique integer called *nameid*. To maintain this, a B^+-Tree, named *NameTree,* with the search key being *nameid* is employed. At each entry of a leaf node of *NameTree*, instead of the links of records, there are two links to two new B^+-Trees having *value* and (*parent, value*) as their search keys, respectively. We call these trees *ValueTree* and *ParentTree*. The leaves of *ValueTree* and *ParentTree* store links to *a-node* or *t-node* records (data records). Combining these three trees, we have a Nested B^+-Tree (NBT) as shown in Figure 11.

Second, based on the idea of the MHT, the NBT is attached with more information to facilitate the proof of query assurances. First, each node keeps hashes of its children:

a-node:	$H_{a\text{-}node} = h(nodeid\|\|xtype\|\|...\|\|value)$
t-node:	$H_{t\text{-}node} = h(h(nodeid\|\| ...\|\|value)\|\|\cup_i H_{attr})$
Leaf of *ValueTree*, *ParentTree*:	$H_L = h(\cup_i H_{data\text{-}record})$
Internal node:	$H_I = h(\cup_i H_{child\text{-}node})$
Leaf of *NameTree*:	$H_{L\text{-}N} = h(H_{vtree} \|\| H_{ptree})$
Root of *NameTree*:	$H_R = h(\varepsilon \|\| \cup_i H_{child\text{-}node})$

where H_{attr} is hash value of an *a-node* of a given *t-node*; $H_{data\text{-}record}$ is either $H_{a\text{-}node}$ or $H_{t\text{-}node}$ that associated to the link; $H_{child\text{-}node}$ is one of H_L, H_I or $H_{L\text{-}N}$; H_{ptree} and H_{vtree} are hash values of *ParentTree*'s

and *ValueTree*'s roots; ε denotes a timestamp value and *h()* is a non-invertible hashing function. Additionally, *NameTree*'s root is signed by the outsourcer's private key and the corresponding public key and timestamp ε are communicated to all valid users. With these settings, the resulting tree, named the *Nested Merkle B$^+$-Tree*, is capable of ensuring query assurance objectives. A concrete example below makes this clearer.

Figure 11. Nested B+-Tree.

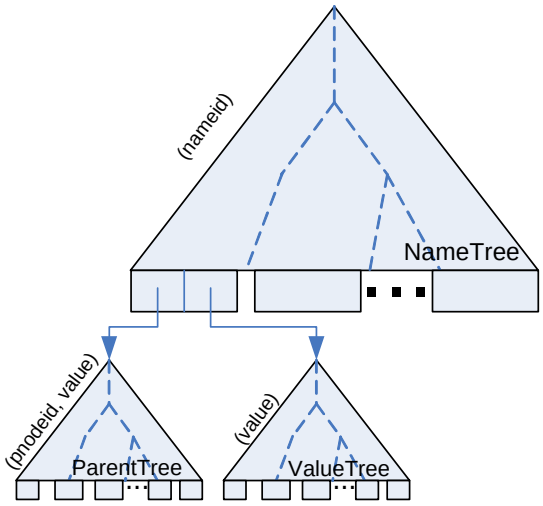
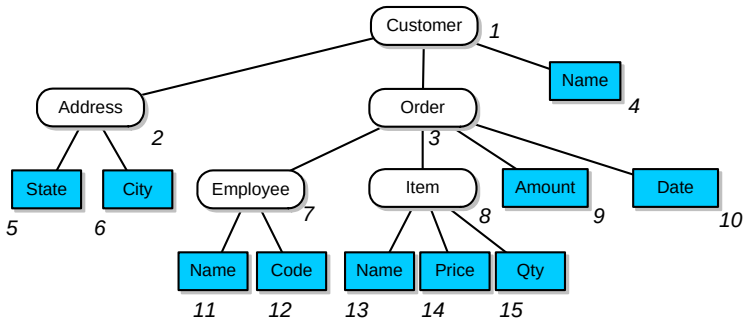


Figure 12 illustrates an example of a labeled XML schema tree, where round, rectangular and sharp corner nodes denote *elements* and *attributes*, respectively, of the XML document. The number next to a node is *nameid* value. Assume a user poses a query to the service provider: "List all sold items named 'TV'". The corresponding query in XPath is */Customer/Order/Item[@name ="TV"]*. To answer this query, the server should process the mapped query (*nameid=13, value='TV'*). To do this, the server scans *NameTree* with *nameid=13* to get the *ValueTree*, then it scans on the found *ValueTree* with *value='TV'* to list out all satisfied attributes *name_13* and builds a *verification object* (VO) for authenticity. With the *pnodeid* field in each found *name_13*, it reads and appends these *Item_8* into the VO. Because there is only one *Item_8* for each *name_13*, the server does not need to provide any information to prove query assurance. For each *Item_8*, it returns two remain attributes.

Figure 12. An example of labeled XML schema tree.



Note that, although user privacy can be incorporated into this approach by employing, for example, techniques proposed in (Lin & Candan, 2004a; Dang, 2005; Dang, 2008), this approach supports only the SS model and data privacy has not been discussed. This aspect is also typical of other hybrid tree-based approaches like (Jammalamadaka & Mehrotra, 2006). This is quite a similar approach with different auxiliary information being embedded in the outsourced XML tree to facilitate several query types. Although data confidentiality is ensured, this approach does not provide query assurances. Even so, there are two notable points of this approach: (1) the proposal of a set of encryption primitives using which a client can propose fairly complex set of security policies on XML documents; (2) the introduction of a novel multi-dimensional partitioning strategy that allows query processing to take place at the server side and overcomes the limitations of single-dimensional partitioning techniques.

3.7 Other Approaches

First of all, in (Brinkman et. al., 2004) the authors, inspired by an approach in (Song, Wagner, & Perrig, 2000), proposed a method based on the linear search algorithm to apply queries on encrypted XML documents. Basically, Song et al. (2000) introduced a way to search for the existence of a word in an encrypted textual document. However, it does not scale well for large databases because the performance is linear in the document size. Based on this idea, Brinkman et al. (2004) developed a tree search algorithm applicable to encrypted XML databases. The newly proposed algorithm is more efficient in this specific context since it exploits the XML tree structure. Experiments showed that the encryption speed of the new algorithm remains linear in the input size, but that a major improvement in the search speed can be achieved. Nevertheless, this new algorithm needs to store a table containing structural information of the XML database at the server, thus compromising data confidentiality and opening a backdoor to inference attacks. Besides, it does not support flexible levels of encryption granularity.

Furthermore, there are three other previous works in the literature addressing queries over encrypted XML documents (Schrefl et. al., 2005; Yang et. al., 2006; Wang & Lakshmanan, 2006). First, Schrefl et al. (2005) proposed a technique that allows XPath selection queries to be executed at the server. Their technique supports both static and dynamic documents but it incurs two crucial limitations (Jammalamadaka & Mehrotra, 2006): (1) cannot support range queries; and (2) requires multiple rounds of communication between the client and server to answer a single query, thereby potentially undermining the performance. Next, Yang et al. (2006) presented XQEnc and XML encryption technique based on vectorization and skeleton compression of XML paths. The proposed XML encryption technique is in accordance with W3C standards (W3C, 2008). To facilitate query processing, the authors proposed using any of the existing techniques such as single-dimensional partitioning or order-preserving encryption. Last, Wang and Lakshmanan (2006) introduced a scheme for storing the meta-data on the server to support efficient query processing but, as pointed out by the authors, their approach still incurs performance penalties and is vulnerable to inference attacks. Moreover, this approach is also not cost-efficient for supporting dynamic outsourced XML databases.

We must emphasize that all the four approaches above and most of the introduced ones in previous sections belong to the SS model. Therein, security requirements are very much easier to address comparing with those of the SMS and MMS models (see Table 1). Therefore, much work still remains to be done in order to make outsourced XML databases practical. We succinctly point out major open issues and future trends in the next section.

4. OPEN ISSUES AND FUTURE TRENDS

Clearly, we are at the initial stage of the research for resolving security issues in outsourced XML databases. In order to apply the research results to real-world applications, we will need more time (Evdokimov et. al., 2006; Wang & Lakshmanan, 2006). Our discussions in previous sections have demonstrated that none of the existing approaches has dealt radically with all crucial security issues concerning its model. Hence, there are still a vast number of security- and privacy-related issues/protocols that need to be solved. We introduce open research issues based on the five well-defined security problems of outsourced XML databases as follows:

- *Data confidentiality*: As discussed, the use of certain data encryption schemes for the outsourced database has drawn much more attention in comparison with a multiple service provider-based solution for data distribution. It is clear that this choice is natural and reasonable when developing a practical ODBS/OXMLDBS model. Nevertheless, currently existing encryption schemes, including *all* state-of-the-art symmetric and asymmetric encryption algorithms, create many obstacles and limitations for the querying process and for ensuring security requirements like those in in-house database services. *Is there any completely new encryption scheme that fits well with the hosting outsourcing model?* This big challenging question is now open to the research community. *This new encryption scheme should be able to facilitate both the confidentiality of the outsourced database and an effective and efficient querying method*. To the best of our knowledge, no previous or ongoing work has focused on this direction. Note that, however, if such an encryption algorithm were devised, the rest of all implementations for OXMLDBS models would be very much simpler.
- *User and data privacy*: Assuming that we are still using the state-of-the-art encryption schemes because an encryption algorithm ideal for the aforementioned OXMLDBS models does not exist, user privacy is not a big issue. The research results have shown that *user privacy* can be assured at a computationally secure level, and this is enough for most real-world applications. Notably, this problem can be solved without special hardware supports. On the other hand, current solutions to *data privacy* in OXMLDBS models have less incentive. Some of the typical existing solutions like (Dang, 2006b; Smith & Safford, 2001) must resort to special hardware or a TTP to mitigate the complexity of this very challenging issue. Therefore, there are two issues that need to be further investigated: (1) studying new protocols/techniques for ensuring data privacy in OXMLDBS models; and (2) addressing technological issues with current special hardware, say IBM SCs, in order for them to be viable and applicable. As pointed out in previous work such as (Smith & Safford, 2001; Mykletun & Tsudik, 2005), a secure coprocessor is somewhat limited in its computing resources, both in processor speed and the amount of on-board memory available. This is a consequence of a variety of factors, predominantly that of heat dissemination. For instance, an IBM 4758 SC, at the beginning, is equipped with a 99 MHz processor and 2 MB on-board memory only (IBM, 2008b). With the continued growth of the hardware technology, we are expecting a brighter future for such special hardware. Again, with the future technologies, what does a TTP looks like? If TTPs can be developed effectively, data privacy and many other challenging security issues will also be able to be resolved much easier and more cost-efficiently.
- *Query assurance*: Clearly, this problem has not been studied extensively for outsourced XML databases. Ensuring the correctness of query answers, however, has not been a big problem, but completeness- and freshness-related aspects are more challenging. Existing solutions still incur the severe limitation of having heavy computation on the client side. This reduces the benefits

that the OXMLDBS models offer. Therefore, new protocols/techniques to address the completeness and freshness of query results will be of great interest in the future. Many new related problems are also foreseen (Dang, 2008), such as the *over redundancy* problem whereby the server sends the user more than what should be returned in the answers. This may cause a user to pay more for the communication cost, to incur higher computation costs, and so this issue (and many others) needs to be investigated carefully.

- *Secure auditing*: Except for (Dang, 2006b), which must resort to a TTP to accomplish this challenging task, none of the previous research work has addressed this vital security issue, even though it is not new. Much more work will need to be done in the future to solve this issue. Moreover, the problem is exacerbated increasingly due to the new rules related to *secure auditing and compliance* such as SOX, HIPAA, PCI, etc. (Dang et. al. 2007; Jaquith, 2007). In complying with these rules, companies are forced to deal with new aspects of the expanding digital universe – since the rules set standards for record keeping, records retention, information security, and privacy protection, among other things. New rules for things like the legal discovery of documents, called "e-discovery," are driving companies to formalize new records management policies, develop archiving standards, and institute policy changes and employee training (Gantz, 2007; Dang et. al. 2007). These new requirements will significantly affect the problem with which we are concerned and will undoubtedly exacerbate it.
- *Secure and efficient storage*: This issue is totally new to outsourced XML databases. However, it is not difficult to understand because a cost-effective model for secure and efficient storage of *unencrypted* XML data has never been created.

Apart from the individual security requirement as shown above, we will also have to solve the very difficult problem of how to develop a fully-fledged solution to satisfy integrated assurances (e.g., confidentiality, privacy of access, and query assurance all together) for each concerned OXMLDBS model. Besides, consolidating and further developing the existing research results to facilitate arbitrary operations on outsourced XML databases like those of the in-house DBMSs (Evdokimov & Günther, 2007; Mykletun & Tsudik, 2006) will also be among the most challenging of future research activities. Addressing all of these problems is the main goal because it will make the OXMLDBS models practical.

In addition, it is not difficult to guess that future trends of the data outsourcing paradigm will not only focus on outsourcing RDBMS or XML database services, but also on other large file systems like huge streaming databases, for example. This will, in turn, need new outsourcing models, new querying and security protocols to meet data management needs as effectively and efficiently as can an in-house service. The increasing importance of XML data in emerging applications, such as SOA-based systems, web services, e-business, etc., is a solid and well-grounded basis to consolidate our aforementioned prediction. Going along with this trend, it is foreseen that hardware-based solutions will become much more popular among end users because of their inherent natural simplicity.

5. CONCLUSION

In this chapter, we conducted an in-depth discussion of security issues in outsourced XML databases, and proposed several related future research directions. Because the outsourcing of XML databases is only a sub-problem of a very much broader field, namely the outsourcing of database services, we firstly presented background knowledge, security issues, and related work in the area of ensuring data

outsourcing security. Moreover, because most existing approaches to security issues in outsourced XML databases are based on those of the broader field, we have also clearly defined and classified security issues into five major issues: data confidentiality, user and data privacy, query assurance, secure auditing, secure and efficient storage model. On this basis, we next presented and discussed the state-of-the-art approaches to deal with security issues for outsourced XML databases in four typical OXMLDBS models. We have categorized these approaches into seven classes: (1) authentic publication of XML documents: a semi-OXMLDBS model; (2) oblivious operations on outsourced search tree; (3) secure multi-party computation-based approaches; (4) trusted third party-based approaches; (5) hardware-based approaches; (6) hybrid tree-based approaches; and (7) other approaches. With each class, technical details of the related approaches were introduced, and real-world case studies and intuitive examples were used to clarify the discussions. We have emphasized that most existing approaches focus on the SS model (see Figure 3) and none of them has radically addressed related security issues.

We introduced open research issues relevant to the problem of concern in section 4. Therein, we also provided general discussions of future trends that have been identified from the previous works, and the needs of emerging application domains. Finally, we must conclude that research results of dealing with security issues in outsourced XML databases are currently too immature for practical purposes. Many more research activities will have to be carried out to make the outsourced XML database service model viable in practice.

REFERENCES

Aggarwal, G., Bawa, M., Ganesan, P., Garcia-Mollina, H., Kenthapadi, K., Motwani, R., Srivatsava, U., Thomas, D., & Xu, Y. (2005). Two can keep a secret: A distributed architecture for secure database services. In *CIDR,* pages 186-199.

Asonov, D. (2001). Private information retrieval: An overview and current trends. In *ECDPvA,* pages 889-894.

Axelrod, C.W. (2004). *Outsourcing information security*. Norwood, MA: Artech House.

Bertino, E., Carminati, B., Ferrari, E., Thuraisingham, B., & A. Gupta (2004). Selective and authentic third-party distribution of XML documents. In *IEEE Transactions on Knowledge and Data Engineering*, *16*(10), pages 1263-1278.

Boneh, D., Crescenzo, G., Ostrovsky, R., & Persiano, G. (2004). Public key encryption with keyword search. In *EUROCRYPT,* pages 506-522.

Brinkman, R., Doumen, J., & Jonker, W. (2004). Using secret sharing for searching in encrypted data. In *SDM*, pages 18-27.

Burmester, M., Desmedt, Y., Wright, R.N., & Yasinsac, A. (2004). Accountable privacy. In *International Workshop on Security Protocols,* pages 83-95.

Brinkman, R., Feng, L., Doumen, J., Hartel, P.H., & Jonker, W. (2004). Efficient tree search in encrypted data. In *Information System Security Journal 13*, pages 14-21.

Bertino, E., Ferrari, E., Paci, F., & Provenza, L.P. (2007). A system for securing push-based distribution of XML documents. In *International Journal of Information Security 6*(4), pages 255-284.

Boneh, D., Gentry, C., Lynn, B., & Shacham, H. (2003). Aggregate and verifiably encrypted sig¬natures from bilinear maps. In *EUROCRYPT,* pages 416-432.

Bouganim, L., & Pucheral, P. (2002). Chip-secured data access: Confidential data on untrusted servers. In *VLDB,* pages 131-142.

Bostick, J. (2008). *Why you should outsource your database infrastructure*. Retrieved February 16, 2008 from http://www.sourcingmag.com/content/c070620a.asp.

Brinkman, R., Schoenmakers, B., Doumen, J., & Jonker, W. (2005). Experiments with queries over encrypted data using secret sharing. In *SDM,* pages 33-46.

Carminati, B., & Ferrari, E. (2006). Confidentiality enforcement for XML outsourced data. In *EDBT Workshop*, pages 234-249.

Castano, S., Fugini, M.G., Martella, G., & Samarati, P. (1995). *Database security*. Boston: Addison-Wesley/ACM Press.

Chor, B., Goldreich, O., Kushilevitz, E., & Sudan, M. (1995). Private information retrieval. In *FOCS*, pages 41-50.

Dang, T.K. (2003). *Semantic based similarity searches in database systems: Multidimensional access methods, similarity search algorithms*. PhD Thesis, FAW-Institute, University of Linz, Austria.

Dang, T.K. (2005). Privacy-preserving search and updates for outsourced tree-structured data on untrusted servers. In *iTrust,* pages 338-354.

Dang, T.K. (2006a). A practical solution to supporting oblivious basic operations on dynamic outsourced search trees. In *International Journal of Computer Systems Science and Engineering 21*(1), pages 53-64.

Dang, T.K. (2006b). Security protocols for outsourcing database services. In *Information and Security: An International Journal 18*, pages 85-108.

Dang, T.K. (2008). Ensuring correctness, completeness and freshness for outsourced tree-indexed data. In *Information Resources Management Journal 21*(1), pages 59-76.

Dang, T.K., Son, N.T. (2006). Providing query assurance for outsourced tree-indexed data. In *HPSC,* pages 207-224.

dbaDirect Company (2008). Retrieved from http://www.dbadirect.com.

Devanbu, P.T., Gertz, M., Kwong, A., Martel, C., Nuckolls, G., & Stubblebine, S.G. (2001). Flexible authentication of XML documents. In *CCS*, pages 136–145.

Dang, T.K., Le, T.T.H, & Truong, D.T. (2007). An extensible framework for database security assessment. In *iiWAS*, pages 419-425.

Dang, T.K., Truong, Q.C., Cu-Nguyen, P-H., & Tran-Thi, Q-N. (2008). An extensible framework for detecting database security flaws. In *ACOMP,* pages 68-77.

Du, W., & Atallah, M.J. (2000). Protocols for secure remote database access with approximate matching. In *ACMCCS Workshop*.

Damiani, E., Vimercati, S.D.C., Jajodia, S., Paraboschi, S., & Samarati, P. (2003). Balancing confidentiality and efficiency in untrusted relational DBMSs. In *ACMCCS*, pages 93-102.

Evdokimov, S., Fischmann, M., & Günther, O. (2006). Provable security for outsourcing database operations. In *ICDE*, page 117..

Evdokimov, S. & Günther, O. (2007). Encryption techniques for secure database outsourcing. In *ESORICS*, pages 327-342.

Fredkin, E. (1960). Trie memory. In *Communications of the ACM 3*(9), pages 490–499.

Gantz, J.F. (2007). *The expanding digital universe: A forecast of worldwide information growth through 2010*. IDC White Paper. Retrieved February 17, 2008 from http://www.emc.com.

Gertner, Y., Ishai, Y., Kushilevitz, E., & Malkin, T. (1998). Protecting data privacy in private information retrieval schemes. In *STOC*, pages 151-160.

Goldreich, O., & Ostrovsky, R. (1996). Software protection and simulation on oblivious RAMs. In *Journal of the ACM 43*, pages 431–473.

Hacigümüs, H., Iyer, B.R., Li, C., & Mehrotra, S. (2002). Executing SQL over encrypted data in the database-service-provider model. In *SIGMOD*, pages 216-227.

Hacigümüs, H., Mehrotra, S., & Iyer, B.R. (2002). Providing database as a service. In *ICDE*, pages 29-40.

IBM IT Outsourcing and Hosting (2008a). Retrieved from http://www.ibm.com/services/stratout.

IBM PCI Cryptographic Coprocessor (2008b). Received from http://www-03.ibm.com/security/cryptocards/pcicc/overview.shtml on March 8, 2008.

Iyer, B.R., Mehrotra, S., Mykletun, E., Tsudik, G., & Wu, Y. (2004). A framework for efficient storage security in RDBMS. In *EDBT*, pages 147-164..

Jammalamadaka, R.C., & Mehrotra, S. (2006). Querying encrypted XML documents. In *IDEAS*, pages 129-136.

Jaquith, A. (2007). *Security metrics: Replacing fear, uncertainty and doubt*. Addison-Wesley Professional.

Li, F., Hadjieleftheriou, M., Kollios, G., & Reyzin, L. (2006). Dynamic authenticated index structures for outsourced databases. In *SIGMOD*, pages 121-132.

Lin, P., & Candan, K.S. (2004a). Hiding traversal of tree structured data from untrusted data stores. In *ICEIS Workshop*, pages 314-323.

Lin, P., & Candan, K.S. (2004b). Secure and Privacy Preserving Outsourcing of Tree Structured Data. In *SDM*, pages 1-17.

Merkle, R.C. (1980). Protocols for public keys cryptosystems. In *Symposium on Research in Security and Privacy*, pages 122-134.

Mykletun, E., Narasimha, M., & Tsudik, G. (2004). Authentication and integrity in outsourced databases. In *NDSS*.

Mykletun, E., &Tsudik, G. (2005). Incorporating a secure coprocessor in the database-as-a-service model. In *IWIA,* pages 38-44.

Mykletun, E., & Tsudik, G. (2006). Aggregation queries in the database-as-a-service model. In *Data and Applications Security*, pages. 89-103.

Natan, R.B. (2005). Implementing database security and auditing. Digital Press.

Narasimha, M., & Tsudik, G. (2006). Authentication of outsourced databases using signature aggregation and chaining. In *DASFAA*, pages 420-436.

Nguyen, V.H., Dang T.K., Son, N.T., & Kueng, J. (2007). Query assurance verification for dynamic outsourced XML databases. In *FARES,* pages 689-696.

Nguyen, V.H., & Dang, T.K. (2008). A novel solution to query assurance verification for dynamic outsourced XML databases. In *International Journal of Software 3*(4), pages 9-16.

Pang, H.H., & Tan, K-L. (2004). Authenticating query results in edge computing. In *ICDE*, pages 560-571.

Pang, H.H., Jain, A., Ramamritham, K., & Tan, K-L. (2005). Verifying completeness of relational query results in data publishing. In *SIGMOD*, pages 407-418.

PRIME Project (2004). *The PRIME Project: Privacy and identity management for Europe*. Retrieved from https://www.prime-project.eu.

R-Tree Portal (2006). Received from http://www.rtreeportal.org/spatial.html.

Schrefl, M., Grün, K., & Dorn, J. (2005). SemCrypt-Ensuring privacy of electronic documents through semantic-based encrypted query processing. In *ICDE Workshop*, page1191.

Sion, R. (2005). Query execution assurance for outsourced databases. In *VLDB,* pages 601-612.

Smith, S.W., & Safford, D. (2001). Practical server privacy with secure coprocessors. In *IBM Systems Journal 40*(3), pages 683-695.

Song, D.X., Wagner, D., & Perrig, A. (2000). Practical techniques for searches on encrypted data. In *IEEE Symposium on Security and Privacy*, pages 44-55.

Thuraisingham, B. (2005). *Database and applications security: Integrating information security and data management*. Auerbach Publisher.

Umar, A. (2004). *Information security and auditing in the digital age: A managerial and practical perspective*. NGE Solutions.

W3C-World Wide Web Consortium (2008). Retrieved from http://www.w3c.org.

Wang, H., Lakshmanan, L.V.S. (2006). Efficient secure query evaluation over encrypted XML databases. In *VLDB*, pages 127-138.

Xie, M., Wang, H., Yin, J., & Meng, X. (2007). Integrity auditing in outsourced data. In *VLDB*, pages 782-793.

Yang, Y., Wilfred, N., Lau, H.L., & Cheng, J. (2006). An efficient approach to support querying secure outsourced XML information. In *CAiSE*, pages 157-171.